

Quelques algorithmes de tri

BUT Informatique - S5 - Algorithmique

1. Autour du tri par insertion

- Implanter (*sans regarder le cours ni une solution sur internet!*) une méthode `tri_insertion` pour trier dans l'ordre **croissant** un tableau d'entiers en suivant l'algorithme de tri par insertion. On fera en sorte que le tableau à trier soit initialisé avec des entiers pris dans 0, 1, 2, ... 100 de façon aléatoire.
- Implanter la méthode `tri_insertion_dec` qui trie dans l'ordre **décroissant**.
- Implanter la méthode `tri_insertion_dic` qui améliore l'efficacité de la méthode `tri_insertion` en utilisant une méthode `recherche_rang` qui procède comme une recherche dichotomique pour trouver la position dans la partie triée du tableau où chaque nouvel entier doit être inséré.
- Quelle est la complexité (dans le pire cas) de cette nouvelle version du tri par insertion ?

1.1. Code `solution`

```
1: public static void tri_insertion(Integer[] tab){
2:     for (int j = 1; j<tab.length; j++){
3:         int key = tab[j];
4:         int i = j-1;
5:         while (i>-1 && tab[i]>key) {
6:             tab[i+1] = tab[i];
7:             i--;
8:         }
9:         tab[i+1] = key;
10:    }
11: }
12:
13: public static void tri_insertion_dec(Integer[] tab){
14:     for (int j = 1; j<tab.length; j++){
15:         int key = tab[j];
16:         int i = j-1;
17:         while (i>-1 && tab[i]<key) {
18:             tab[i+1] = tab[i];
19:             i--;
20:         }
21:         tab[i+1] = key;
22:     }
23: }
24:
25: public static int recherche_rang(Integer e, int deb, int fin, Integer[] tab){
26:     /* hypothèse: tab est trié dans l'ordre croissant */
27:     int milieu = deb + (fin-deb)/2;
28:     while (deb!=fin){
29:         if (e>tab[milieu]) deb = milieu+1;
30:         else fin = milieu;
31:         milieu = deb + (fin-deb)/2;
32:     }
33:     return (tab[deb])>e?deb:deb+1;
34: }
35:
36: public static void tri_insertion_dichotomique(Integer[] tab){
37:     for (int j = 1; j<tab.length; j++){
38:         int key = tab[j];
39:         int k = recherche_rang(tab[j],0,j-1,tab);
40:         for (int i = j-1; i>=k; i--)
41:             tab[i+1]=tab[i];
42:         tab[k] = key;
43:     }
44: }
```

1.2. Complexité `solution`

La complexité, dans le pire des cas, est toujours $\Theta(n^2)$. En effet, si le nombre de comparaisons est en $\Theta(n\log(n))$, le nombre d'affectations reste en $\Theta(n^2)$. Remarque: Dans la version précédente de l'algorithme l'insertion il y a toujours $\Theta(n^2)$ comparaison mais on pourrait réécrire l'algo pour déplacer les éléments sans comparaison.

2. Tri par sélection

- Le tri par sélection consiste à trouver le plus petit élément du tableau et à le permuter avec l'élément se trouvant à la première position. On recommence avec le 2ème plus petit élément qu'on permute avec la deuxième position et ainsi de suite avec les éléments suivants. Ainsi, au départ la partie non triée du tableau correspond au tableau entier puis elle se rétrécit jusqu'à ne contenir que la dernière case du tableau. Implémentez en java la méthode `tri_selection`.
- Donnez la complexité de cet algorithme dans le pire des cas et dans le meilleur des cas.

2.1. Code `solution`

```
1: public static void tri_selection(Integer[] tab){
2:     for (int i = 0; i < tab.length-1; i++){
3:         int min = i;
4:         for (int j = i+1; j < tab.length; j++)
5:             if (tab[j] < tab[min])
6:                 min = j;
7:         int tmp = tab[min];
8:         tab[min] = tab[i];
9:         tab[i] = tmp;
10:    }
11: }
```

2.2. Calcul de complexité `solution`

Dans le pire cas, le temps d'exécution est:

$$T(n) = nc_2 + (n-1)c_3 + c_4 \sum_{i=0}^{n-1} t_i + c_5 \sum_{i=0}^{n-1} (t_i - 1) + c_6 \sum_{i=0}^{n-1} t'_i + (n-1)(c_7 + c_8 + c_9)$$

où t_i est le nombre de fois que la boucle `for` sur `j` est réalisée à l'itération i de la boucle `for` englobante. Notez que la ligne 6 n'est exécutée que si la condition est `tab[j]<tab[min]` est satisfaite, c'est-à-dire un nombre de fois t'_i satisfaisant $0 \leq t'_i \leq t_i - 1$ (car, au plus, l'affectation est réalisée autant de fois que la ligne 5). Dans tous les cas, on a $t_i = n - i$ et dans le pire cas, $t'_i = t_i - 1$ et donc

$$\sum_{i=0}^{n-1} t_i = \sum_{i=0}^{n-1} (n - i) = n^2 - \sum_{i=0}^{n-1} i = n^2 - \frac{n(n-1)}{2} = \frac{n(n-1)}{2} \text{ et } \sum_{i=0}^{n-1} (t_i - 1) = \frac{(n-2)(n-1)}{2}$$

Par conséquent, $T(n) \in O(n^2)$ dans le pire cas.

Dans le meilleur cas, le tableau est trié et l'affectation à la ligne 6 n'est jamais réalisée (ie. $t'_i = 0$). Le terme $c_6 \sum_{i=0}^{n-1} (t_i - 1)$ dans l'expression de $T(n)$ est supprimé mais pas le terme $c_4 \sum_{i=0}^{n-1} t_i$, donc le coup reste quadratique, c'est-à-dire $T(n) = \Omega(n^2)$.

2.3. Pour aller plus loin: preuve de correction... `solution`

Soit l'invariant:

$Inv(i)$ = Le sous-tableau $tab[0, \dots, i-1]$ est trié et tous ses éléments sont plus petits que ceux du sous-tableau $tab[i, \dots, n]$.

où n est la longueur du tableau tab . Montrons qu'il satisfait les 3 propriétés:

- **initialisation** $Inv(0)$: C'est trivialement satisfait car $tab[0, \dots, -1]$ est vide.
- **conservation**: Supposons que $Inv(i)$ est satisfait au début de la boucle `for` sur la variable `i`, montrons que $Inv(i')$, où i' est la valeur de la variable `i` à la fin du bloc d'iteration, est aussi satisfait. L'itération recherche le plus élément de $tab[i, \dots, n]$ dont on sait, par $Inv(i)$, qu'il plus petit que tous les éléments de $tab[0, \dots, i-1]$. Cet élément étant permuté avec $tab[i]$, il en résulte que $tab[0, \dots, i]$ est trié et tous ces éléments sont plus petits que $tab[i+1, \dots, n]$. En toute fin d'itération la variable `i` a pour valeur $i' = i + 1$, ce qui montre que $Inv(i')$ est satisfait.
- **terminaison**: à l'issue de la dernière itération, la variable `i` a pour valeur $n-1$ et, par conservation, $Inv(n-1)$ est satisfait, c'est-à-dire le sous-tableau $tab[0, \dots, n-2]$ est trié et ses éléments sont plus petits que le dernier élément $tab[n-1]$. Par conséquent, le tableau tab est entièrement trié.□

3. Tri à bulles

Implantez les deux algorithmes de tri à bulle vus en TD: `void bubbleSort(int [] tab)` et la version optimisée `static void bubbleSortOpt(int [] tab)`.

3.1. `solution`

```
static void swap(int [] tab, int idx, int idx2){
    int tmp = tab[idx];
    tab[idx] = tab[idx2];
    tab[idx2] = tmp;
}

static void bubbleSort(int [] tab){
    for(int i=tab.length-1; i>0; i--){
        for(int j=0; j<i; j++){
            if(tab[j]>tab[j+1]){
                swap(tab,j,j+1);
            }
        }
    }
}

static void bubbleSortOpt(int [] tab){
    int end = tab.length - 1;
    while(end > 0){
        int perm = -1;
        for(int i=0; i<end; i++){
            if(tab[i]>tab[i+1]){
                swap(tab,i,i+1);
                perm = i;
            }
        }
        end = perm;
    }
}
```

4. Comparaison des algorithmes

- Ajoutez aux méthodes précédemment implantées des compteurs permettant de calculer le nombre total de comparaisons et d'affectations réalisées à chaque tri. Pour obtenir le coût maximale, générez des tableaux triés par ordre décroissants de tailles 10, 100, 1000 et 10000, et triez-les avec les algorithmes implantés. Remplissez un tableau avec les valeurs des compteurs obtenus pour les différents algorithmes et tailles de tableaux.
- Pour évaluer le coût moyen, générez 1000 tableaux aléatoires pour chaque taille prédéfinie, puis calculez le nombre moyen d'opérations effectuées à chaque fois. Récapitulez vos résultats sous forme de tableaux comme à la question précédente.

Auteur: Cédric Lhoussaine (notamment inspiré d'exercices du livre *Algorithmique* de Cormen, Leiserson, Rivest & Stein. et d'autres enseignants du BUT)